

Curso Inicial Front-End

Clase 05: Introducción a CSS: Estilo y Cascada

Módulo 1: Arquitectura y Maquetación Moderna (HTML y CSS)



Presentación

Es hora de darle estilo a la web. En esta clase conoceremos **CSS** (Cascading Style Sheets), el lenguaje que permite transformar un documento **HTML** plano en una experiencia visual atractiva. Entenderemos las reglas de la **Cascada** y dominaremos el **Box Model**, el concepto más importante para entender cómo se posicionan los elementos en la pantalla.



Objetivos

- Dominar los selectores básicos y entender la jerarquía (cascada) de **CSS**.
- Comprender y manipular el **Box Model** (Content, Padding, Border, Margin).
- Aplicar efectos visuales modernos como sombras (**box-shadow**), bordes redondeados y gestión de desbordamiento (**overflow**).



Bloques Temáticos

- Tema 1: Selectores, colores y tipografías.
 - ¿Qué es **CSS**?
 - Los 3 caminos del **CSS**: ¿Dónde escribo? y Puesta en Marcha
 - Selectores: ¿A quién le aplicamos el estilo?
 - La Guerra de las Unidades: **rem** vs **em** vs **px**
 - Propiedades de Texto y Color
- Tema 2: La cascada, especificidad y herencia.
 - Herencia: ¿Qué se pasa a los hijos genéticamente?
 - El Modelo de Caja (Box Model): La Analogía del Cuadro

En las clases anteriores construimos los cimientos de nuestra casa usando **HTML** semántico. Pero si abres tu archivo **index.html** ahora mismo, notarás que todo está apilado hacia la izquierda, en blanco y negro, y con una fuente aburrida. Es una casa funcional, pero nadie querría vivir ahí.

Hoy entraremos en la etapa de "Diseño de Interiores". Daremos vida a nuestro **HTML** aprendiendo cómo seleccionar elementos y aplicarles estilos siguiendo las reglas del **Modelo de Caja**, el concepto más crítico y fundamental de todo el diseño web.

Tema 1: Selectores, colores y tipografías.

1. ¿Qué es CSS?

CSS significa **Cascading Style Sheets** (Hojas de Estilo en Cascada). Es un lenguaje de diseño (no de programación lógica) que le da al navegador las instrucciones visuales: colores, tamaños, posiciones y animaciones.

- **Hojas de Estilo:** Es un documento especial (con extensión `.css`) donde guardamos todas las reglas de belleza de nuestra web.
- **Cascada (El concepto rey):** Imagina una cascada de agua que cae desde la cima de una montaña. En **CSS**, las reglas fluyen de arriba hacia abajo. Si hay dos instrucciones contradictorias para pintar el mismo botón, el navegador resolverá la pelea siguiendo una jerarquía estricta (que veremos más abajo).

 **Recurso Oficial:** [MDN Web Docs - Introducción a CSS](#)

CSS: Pros y Contras

 Ventajas (Pros)	 Desventajas (Contras)
Separación de responsabilidades: Mantiene el archivo HTML limpio y enfocado solo en el contenido y la estructura.	Complejidad de Especificidad: Las reglas pueden colisionar y sobreescribirse accidentalmente en proyectos inmensos.

✓ Ventajas (Pros)	✗ Desventajas (Contras)
Reutilización Masiva: Una sola clase (ej: <code>.btn-rojo</code>) escrita una vez, puede pintar 100 botones en 10 páginas distintas al instante.	Mantenimiento: Archivos <u>CSS</u> gigantescos sin buena arquitectura pueden volverse imposibles de leer.

La anatomía matemática de una regla CSS

Para que el navegador nos entienda, el código CSS no usa etiquetas de apertura y cierre como HTML, sino "bloques de reglas":

```
selector {  
  propiedad: valor;  
}
```

1. **Selector:** ¿A quién le hablo? (Ej: "Oye, etiqueta `h1`...").
2. **Propiedad:** ¿Qué le quiero cambiar? (Ej: "...quiero cambiar tu `color`...").
3. **Valor:** ¿Cómo lo quiero? (Ej: "...y quiero que sea `red` (rojo)").

1.2. Los 3 caminos del CSS: ¿Dónde escribo? y Puesta en Marcha

Tienes tres formas de inyectar CSS a tu HTML.

1. **En línea (Inline):** Escribiéndolo directamente adentro de la etiqueta HTML (`<p style="color: blue;">`). ¡**PROHIBIDO!** Ensucia el código HTML y te obliga a repetir la regla en cada párrafo de tu web manualmente.

2. **Interno:** Usando la etiqueta `<style>` adentro del `<head>` del **HTML**. Solo se usa para pruebas rápidas de 5 minutos, nunca en un trabajo real.
3. **Externo (El Estándar Profesional):** Crear un archivo separado exclusivamente para **CSS**.

Puesta en marcha (Sigue estos pasos en VS Code): 1. Abre tu proyecto actual en VS Code. Crea una nueva carpeta llamada `css`.

1. Dentro de esa carpeta, crea un archivo llamado `styles.css`.
2. Ahora, debes decirle a tu `index.html` que ese archivo existe. Ve al `<head>` de tu **HTML** y escribe la etiqueta de enlace (link): `html <!-- Se vincula estrictamente en el <head> del HTML --> <link rel="stylesheet" href="css/styles.css">`
3. *Truco de VS Code:* Haz clic en el icono de "Dividir Pantalla" (Split Editor) arriba a la derecha. Pon tu **HTML** a la izquierda y tu **CSS** a la derecha. ¡Así trabajan los profesionales!

Arquitectura Modular: El poder del `@import`

En proyectos corporativos grandes, tener un archivo `styles.css` de 2000 líneas es una pesadilla de mantener. La solución profesional es **modularizar** tus estilos. Creas archivos pequeños y separados (`header.css`, `footer.css`, `botones.css`) y luego usas un archivo **CSS Centralizador**.

En lugar de poner 10 etiquetas `<link>` sueltas en tu **HTML**, pones **solo una** apuntando a tu `styles.css`. Y dentro de ese `styles.css`, "importas" a todos los demás usando la regla `@import` al mismísimo principio del archivo:

```
/* Archivo: css/styles.css (El Centralizador) */
@import url('./reset.css');
@import url('./header.css');
```

```
@import url('./footer.css');
@import url('./cards.css');

/* A partir de aquí puedes poner estilos globales */
body {
    background-color: #f4f4f4;
}
```

Ventaja Absoluta: Todos tus archivos **HTML** (`index.html`, `contacto.html`) solo necesitan enlazar el `styles.css` único, pero heredan mágicamente todos los diseños importados. ¡Máximo orden y limpieza!



TIP

Truco Profesional (Diferenciar Páginas): Si todos los archivos **HTML** usan el mismo `styles.css`, ¿cómo haces para que el título en `contacto.html` sea rojo y en `index.html` sea azul? ¡Fácil! Le pones una **clase única a la etiqueta** `<body>` de cada archivo.

En tu `contacto.html`: `<body class="page-contacto">` En tu **CSS** global: `.page-contacto h1 { color: red; }`

1.3. Selectores: ¿A quién le aplicamos el estilo?

Para pintar algo, primero debemos "seleccionarlo". Hay tres herramientas básicas:

- **Selector de Etiqueta:** `p { color: red; }` (Afectará a TODOS los párrafos de tu página. Es un bombardeo masivo).
- **Selector de Clase (El más Recomendado):** `.mi-boton { background: blue; }` (Usa un **punto** `.` inicial y afectará a cualquier etiqueta que tenga `class="mi-boton"`. Es un ataque de precisión).
- **Selector de ID (El Único):** `#header { background: black; }` (Usa un **numeral** `#`. Regla de oro: Un ID solo puede usarse UNA VEZ por página **HTML**. Es como un DNI).

Selectores Avanzados (La Familia)

- **Hijo Directo (>):** `ul > li` (Afecta solo a los `li` que están *directamente* dentro de ese `ul`, no a los nietos).
- **Hermano Adyacente (+):** `h1 + p` (Afecta únicamente al primer párrafo que esté pegado *inmediatamente* después del `h1`).
- **Selector de Atributo ([]):** `input[type="text"]` (Afecta solo a los inputs de texto, ignorando a los botones o checkboxes).

Especificidad: El Sistema de Puntos y la Lucha Libre de CSS

Si un elemento tiene estilos enfrentados (ej: el HTML dice clase azul, pero el ID dice rojo), el navegador hace una suma matemática invisible (`0.0.0.0`) para ver quién gana:

- **Estilo en línea (`style="..."`):** Vale 1.0.0.0 puntos (Un misil nuclear. Prácticamente invencible).
- **Selector de ID (#):** Vale 0.1.0.0 puntos (100 puntos).
- **Selector de Clase (.) o Pseudo-clase (:hover):** Vale 0.0.1.0 puntos (10 puntos).
- **Selector de Etiqueta (p, h1):** Vale 0.0.0.1 punto (1 miserable punto).

Ejemplo de Suma Matemática: `#header .nav-item a:hover` => 1 ID (100) + 1 Clase (10) + 1 Etiqueta (1) + 1 Pseudo-clase (10) = **La regla vale 121 puntos.**

La Pelea Final: - `#menu { color: red; }` (Vale 100 puntos) - `.nav-item { color: blue; }` (Vale 10 puntos) - **Ganador:** El menú será **Rojo**, sin importar si la regla azul se escribió en la última línea del archivo. El ID destroza a la Clase.

1.4. La Guerra de las Unidades: rem vs em vs px

Para que un sitio sea accesible en cualquier dispositivo, debes elegir bien con qué regla vas a medir:

Unidad	Tipo	Recomendación de Uso
px (Píxeles)	Absoluta e Inflexible	Úsalos para cosas diminutas como bordes o sombras. ¡Nunca para tipografías! Si un usuario casi ciego sube el zoom de su navegador, los px se negarán a crecer.
rem (Root EM)	Relativa e Inteligente	La mejor para fuentes y márgenes. Se basa en el tamaño del HTML raíz (usualmente 16px). Es predecible y 100% responsiva para el usuario. (Ej: 2rem = 32px).
em (EM)	Relativa Peligrosa	Se basa en el tamaño de fuente de su caja padre inmediata . Si anidas una caja dentro de otra, el tamaño mutará exponencialmente haciéndose gigante. Úsala con precaución.
vh / vw	Viewport (Pantalla)	Basadas en el 1% del alto (vh) o ancho (vw) total de la ventana de Google Chrome. 100vh es ideal para hacer portadas (Hero Sections) que ocupen toda tu pantalla.
%	Porcentaje	Relativo al tamaño de su caja padre. Ideal para maquetar columnas responsivas (ej: width: 50% ocupará la mitad de su contenedor).

1.5. Propiedades de Texto y Color

Para cerrar tu primer diseño, aquí tienes los pinceles básicos para decorar tipografías y fondos:

- **color**: Define el color de la letra.
- **background-color**: Define el color de fondo de la caja.
- **font-family**: Elige el tipo de letra (ej: 'Arial', sans-serif).
- **font-size**: Tamaño de la letra (¡usa siempre rem!).
- **font-weight**: El grosor del trazo (bold para negrita, normal, o números del 100 al 900).
- **text-align**: Alineación horizontal del texto (left, center, right, justify).

Ejemplo Limpio para probar:

```
/* Esto aplicará una fuente moderna y un fondo suave a toda tu web */
body {
  font-family: 'Helvetica', sans-serif;
  color: #333333;
  background-color: #f4f4f4;
}

/* Solo afectará al elemento que tenga class="titulo-principal" */
.titulo-principal {
  font-size: 2.5rem;
  font-weight: 700;
  text-align: center;
  color: #2b6cb0;
}
```

Tema 2: La cascada, especificidad y herencia.

2.1. Herencia: ¿Qué le pasa a los hijos genéticamente?

No todas las propiedades de **CSS** se heredan. Imagina que **CSS** actúa como el ADN:

1. **Propiedades que SÍ heredan (Fuentes y Texto):**
2. **Herencia Automática:** Propiedades como `color`, `font-family` o `text-align` pasan del padre al hijo sin que hagas nada.
3. **Valor `inherit`:** Puedes forzar a un elemento a que copie una propiedad de su padre usando el valor `inherit`. Por ejemplo, si quieres que un botón use el mismo color de texto que el div que lo contiene: `color: inherit;`. Absolutamente todos los textos, títulos y listas de la página nacerán siendo azules y con Arial automáticamente. Se "pasan" de padres a hijos.
4. **Propiedades que NO heredan (El Modelo de Caja):** Si le pones un `border` rojo o un `padding` inmenso a un `div`, sus hijos (párrafos) **NO** nacerán con borde ni padding. Sería un desastre visual si lo heredaran. (Otras no heredables: `margin`, `background`, `width`, `height`).

2.2. El Modelo de Caja (Box Model): La Analogía del Cuadro

Llegamos al concepto definitivo de la web. **TODO** elemento en internet (un texto, un botón, una foto) es físicamente una caja rectangular invisible. Imagina que cada elemento es un **cuadro de pintura colgado en una galería de arte**:

1. **Content (La Obra de Arte)**: Es el texto o la foto real. Es el corazón de la caja.
2. **Padding (El Paspartú)**: Es el cartón blanco que va entre la pintura y el marco de madera. Es un **espacio en blanco interno**.
3. **Border (El Marco de Madera)**: Es la línea física que rodea al cuadro. Puede ser fina, gruesa o totalmente invisible.
4. **Margin (Distancia en la pared)**: Es el espacio vacío de la pared que separa a tu cuadro del cuadro del vecino. Es un **espacio en blanco externo**.

Comparativa Vital: Margin vs Padding

Ambos generan espacio "en blanco", pero equivocarte entre ellos destruirá tu diseño.

Característica	padding (Relleno Interno)	margin (Margen Externo)
Ubicación	Dentro del borde de la caja.	Fuera del borde de la caja.
Color de Fondo	Hereda y toma el color de fondo (background-color) de su propia caja.	Es totalmente transparente (siempre se ve la pared/fondo de la página).
Función Real	Empuja el contenido hacia adentro para que "respire" y no choque con el borde.	Empuja a las cajas "vecinas" hacia afuera para que no se choquen entre sí.

El Reset Profesional y la Matemática de la Caja

Crea una caja mental de ancho `width: 200px` con un `padding: 2px;` y un `border: 5px.`

- **La pesadilla por defecto (`box-sizing: content-box`):** Históricamente, el navegador sumaba todo hacia afuera. Tu caja pasará a medir $200 + 40$ (`padding izquierdo y derecho`) + 10 (`borde izquierdo y derecho`) = ¡`250px reales!`. Esto romperá cualquier estructura de columnas que intentes armar.
- **La Solución Mágica Profesional (`box-sizing: border-box`):** Le dice al navegador: *"Mantén el ancho final clavado en 200px. Mete el padding y el borde HASTA ADENTRO, asfixiando al contenido si es necesario"*.

Tu primer "Reset **CSS**" (Pon esto siempre en la línea 1 de tu `styles.css`):

```
/* El asterisco (*) selecciona absolutamente TODAS las etiquetas de la web
*/
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* Fundamental para que las matemáticas no se
rompan */
}
```

Documentación Oficial (Referencias)

- **CSS:** <https://developer.mozilla.org/es/docs/Web/CSS>
- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>